Homework 2: Q-Learning for Parking Lot Navigation Question Responses

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Problem 1: Tabular Q-Learning

Problem 1a: Environment Representation

The parking lot is modeled as a 7×6 grid (7 rows, 6 columns) matching the layout in Figure 1 of the assignment. The agent starts at the entrance at position $(6, 5)$ and must navigate to one of 8 parking spots. BFS verification confirms optimal paths range from 3 steps (P8, nearest) to 10 steps (P1, farthest).

State space. A state is a tuple (r, c, g) where (r, c) is the car's row and column position and $g \in \{0, \ldots, 7\}$ identifies the target parking spot. The goal is part of the state because the walkable cells change depending on which spot is the target—non-target parking spots act as walls that the car cannot drive over. This means each goal creates a different navigation problem with its own set of reachable cells.

Action space. Four discrete actions: UP (row -1), DOWN (row $+1$), LEFT (col -1), RIGHT (col $+1$). There is no angular steering—the car moves one cell per step in one of the four cardinal directions.

Reward structure. The reward function uses three values:

- $+10.0$ for reaching the goal parking spot (episode terminates)
- -0.1 per valid step (encourages the shortest path)
- -1.0 for hitting a wall, barrier, or grid edge (agent stays in place)

The step penalty pushes the agent toward the BFS-optimal path. The wall penalty discourages the agent from repeatedly bumping into obstacles. The large positive terminal reward makes reaching the goal clearly preferable to wandering.

Barrier modeling. Row 3, columns 1 through 4 are barrier cells. Any action that would move the agent into a barrier cell is treated identically to hitting the grid boundary—the agent stays where it is and receives the -1.0 wall penalty. Columns 0 and 5 of row 3 are open driving lanes, providing passages on both sides of the barrier.

Edge handling. Before executing a move, the environment checks whether the resulting position (r', c') satisfies $0 \leq r' < 7$ and $0 \leq c' < 6$. If the move would take the agent out of bounds, the agent stays in place and receives the -1.0 penalty. The same check applies to barrier cells and non-target parking spots.

Q-table structure. The Q-table is implemented as a Python dictionary mapping $(r, c, g, a) \rightarrow \mathbb{R}$, where $a \in \{0, 1, 2, 3\}$ is the action index. All entries are initialized to 0. Using a dictionary rather than a dense array avoids allocating memory for state-action pairs that are never visited (barrier cells, out-of-bounds states). For the 7×6 grid with 8 goals and 4 actions, the maximum table size is $7 \times 6 \times 8 \times 4 = 1{,}344$ entries, though in practice only the reachable states are populated during training.

Reachability verification. Before training begins, BFS is run from the entrance to every parking spot to confirm all 8 goals are reachable. This catches layout errors early. The BFS shortest path lengths also serve as the ground truth for evaluating whether the learned policies are optimal.

Problem 1b: Q-Learning and Double Q-Learning Algorithms

Q-Learning Algorithm 1 presents the tabular Q-learning algorithm with ε-greedy exploration.

Line 1 initializes the Q-table optimistically at zero. Since most transitions yield negative rewards (-0.1 or -1.0), zero initialization encourages exploration of unvisited states.

Lines 7–11 implement ε-greedy action selection. Early in training, $\varepsilon \approx 1.0$, so the agent explores randomly. As ε decays (line 19), the agent increasingly exploits the learned Q-values.

Lines 14–16 are the core Q-learning update. The key term is $\max_{a'} Q(S', a')$—Q-learning bootstraps from the *best possible* next action regardless of which action was actually taken. This makes Q-learning an *off-policy* algorithm: it learns about the greedy policy while following an exploratory policy.

Line 19 decays ε after each episode. The decay factor of 0.995 means ε reaches approximately 0.01 after 920 episodes, at which point the agent is mostly greedy with occasional random exploration.

Double Q-Learning Algorithm 2 presents the Double Q-learning variant.

How Double Q-Learning improves on Q-Learning. Standard Q-learning suffers from *maximization bias*. The $\max_{a'} Q(S', a')$ term in the update uses the same Q-values

Algorithm 1 Q-Learning with ε-Greedy Exploration

1: Initialize $Q(s,a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
2: Set $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 1.0$, $\varepsilon_{\min} = 0.01$, $\varepsilon_{\text{decay}} = 0.995$
3: **for** each episode $= 1, 2, \ldots, N$ **do**
4: Sample goal g uniformly from $\{0, \ldots, 7\}$
5: $S \leftarrow$ entrance position $(6, 5)$
6: **while** S is not terminal **and** steps $<$ max_steps **do**
7: **if** random() $< \varepsilon$ **then**
8: $A \leftarrow$ random action *// explore*
9: **else**
10: $A \leftarrow \arg\max_a Q(S, a)$ *// exploit*
11: **end if**
12: Take action A, observe reward R and next state S'
13: **if** S' is terminal **then**
14: $Q(S, A) \leftarrow Q(S, A) + \alpha\,[R - Q(S, A)]$
15: **else**
16: $Q(S, A) \leftarrow Q(S, A) + \alpha\,[R + \gamma \max_{a'} Q(S', a') - Q(S, A)]$
17: **end if**
18: $S \leftarrow S'$
19: **end while**
20: $\varepsilon \leftarrow \max(\varepsilon_{\min},\ \varepsilon \cdot \varepsilon_{\text{decay}})$
21: **end for**

both to select the best action and to estimate its value. When Q-values contain noise (which they always do during learning), taking the maximum over noisy estimates systematically overestimates the true value (Van Hasselt 2010).

Double Q-learning fixes this by maintaining two independent Q-tables. On each update step, one table selects the best action (line 9 or 12) while the other evaluates it (line 10 or 13). This decoupling breaks the positive bias because the noise in the selecting table is independent of the noise in the evaluating table.

For action selection during episodes (line 6), the agent uses the average of both tables, which provides a more stable estimate than either table alone.

Empirically, standard Q-learning's mean max Q-values are consistently higher than Double Q-learning's throughout training, confirming the overestimation. In our deterministic environment this bias does not affect the final policy (both algorithms find optimal paths), but in stochastic environments it can lead to suboptimal action selection.

Problem 2: Deep Q-Learning

Problem 2a: Neural Network Design

Inputs. The neural network receives a 4-dimensional vector:

$$\mathbf{x} = \left[\frac{r}{6},\ \frac{c}{5},\ \frac{g_r}{6},\ \frac{g_c}{5}\right] \tag{1}$$

where (r, c) is the car's position and (g_r, g_c) is the goal parking spot's position. All values are normalized to $[0, 1]$ by dividing by the maximum row index (6) and column index (5). The goal position is included as input so a single network can learn policies for all 8 parking spots simultaneously.

Algorithm 2 Double Q-Learning

1: Initialize $Q_1(s,a) = 0$ and $Q_2(s,a) = 0$ for all s, a
2: Set α, γ, ε, $\varepsilon_{\min}$, $\varepsilon_{\text{decay}}$
3: **for** each episode **do**
4: Sample goal g, set $S \leftarrow$ entrance
5: **while** S is not terminal **and** steps $<$ max_steps **do**
6: Choose A via ε-greedy using $\frac{Q_1(S,\cdot)+Q_2(S,\cdot)}{2}$
7: Take action A, observe R, S'
8: **if** random() < 0.5 **then**
9: $A^* \leftarrow \arg\max_a Q_1(S', a)$ *// Q1 selects*
10: $Q_1(S, A) \leftarrow Q_1(S, A) + \alpha[R + \gamma\, Q_2(S', A^*) - Q_1(S, A)]$ *// Q2 evaluates*
11: **else**
12: $A^* \leftarrow \arg\max_a Q_2(S', a)$ *// Q2 selects*
13: $Q_2(S, A) \leftarrow Q_2(S, A) + \alpha[R + \gamma\, Q_1(S', A^*) - Q_2(S, A)]$ *// Q1 evaluates*
14: **end if**
15: $S \leftarrow S'$
16: **end while**
17: Decay ε
18: **end for**

Outputs. The network outputs 4 Q-values, one for each action (UP, DOWN, LEFT, RIGHT). The agent selects the action with the highest Q-value (during greedy execution) or a random action with probability ε (during training).

What the network learns. The network learns the function $Q(s, a; \theta) \approx Q^*(s, a)$—the optimal action-value function. Given a state-goal encoding, it predicts the expected discounted return for each action. Once trained, the greedy policy $\pi(s) = \arg\max_a Q(s, a; \theta)$ should match the optimal BFS policy.

Architecture. The network is a fully connected feedforward network with two hidden layers:

$$\text{Input}(4) \to \text{FC}(128) \to \text{ReLU} \to \text{FC}(128) \to \text{ReLU} \to \text{FC}(4)$$

This architecture was chosen for several reasons. Two hidden layers provide enough capacity to learn the non-linear mapping from positions to Q-values without being so deep that training becomes unstable. The 128-unit width was determined empirically—64 units were insufficient for consistent convergence across all 8 goals, while 128 units achieved 100% success. ReLU activations are standard for DQN and avoid vanishing gradient issues. The total parameter count is approximately 17,540.

Replay memory. A circular buffer (Python `deque`) stores transitions as named tuples $(s, a, r, s', \text{done})$ where s and s' are the 4-dimensional encoded vectors. The buffer has a capacity of 20,000 transitions. When full, new transitions overwrite the oldest ones.

The buffer is filled online during training: every time the agent takes a step in the environment, the resulting transition is pushed into the buffer. Training begins once 64 transitions have accumulated (one full batch). On every subsequent step, a random mini-batch of 64 transitions is sampled from the buffer for one gradient update.

Why replay memory? Without replay, consecutive training samples are highly correlated (adjacent states in the same

episode). This correlation destabilizes neural network training because gradient updates on correlated samples reinforce each other, causing the network to overfit to recent experience and forget earlier lessons. Random sampling from the buffer breaks these correlations, providing the i.i.d.-like samples that gradient descent assumes (Mnih et al. 2015).

Buffer size considerations. A buffer of 20,000 transitions holds roughly the last 2,000–3,000 episodes of experience (at an average of 7 steps per episode once the agent is proficient). This is large enough to decorrelate samples while small enough to keep the data relevant—too large a buffer fills with low-quality early-exploration transitions that slow convergence. We tested sizes from 1,000 to 50,000 and found that 10,000–20,000 provided the best balance.

Target network. A copy of the Q-network (the "target network") provides stable TD targets during training. It is updated every 50 episodes by copying the weights from the online network. Without the target network, the TD target $r + \gamma \max_{a'} Q(s', a'; \theta)$ shifts with every gradient step, creating a moving target that can cause training divergence.

Training details. Adam optimizer, learning rate 5×10^{-4}, MSE loss, batch size 64. Exploration uses ε-greedy with decay factor 0.998 (slower than tabular, because neural convergence benefits from extended exploration). Training runs for 15,000 episodes on the base 7×6 grid.

Problem 2b: Results and Parameter Sensitivity

Learned policies. All three algorithms (Q-learning, Double Q-learning, DQN) achieve 100% success rates across all 8 parking spots. The learned path lengths match the BFS-optimal paths exactly. Table 1 shows the evaluation results.

Table 1: Evaluation results (100 episodes per goal, all three algorithms)

Spot	Success	Avg Steps	Avg Reward	BFS Optimal
P1	100%	10.0	9.10	10
P2	100%	9.0	9.20	9
P3	100%	8.0	9.30	8
P4	100%	5.0	9.60	5
P5	100%	6.0	9.50	6
P6	100%	5.0	9.60	5
P7	100%	4.0	9.70	4
P8	100%	3.0	9.80	3

Figure 1 shows the Q-learning policies for all 8 goals. Each subplot shows the greedy action at every walkable cell for a specific target spot. The arrows consistently point along the BFS-optimal paths.

Figure 1: Q-learning policies for all 8 parking spots. Blue arrows indicate the greedy action at each walkable cell.

Convergence comparison. Figure 2 compares the learning curves of Q-learning and DQN. Q-learning converges in approximately 1,500 episodes, while DQN requires approximately 3,000–6,000 episodes. This is expected—the 7×6 grid has only $\sim$29 walkable states per goal, which is well within tabular capacity. DQN's function approximation adds overhead (replay warm-up, target network lag, gradient noise) that is unnecessary at this scale but becomes essential for larger state spaces.

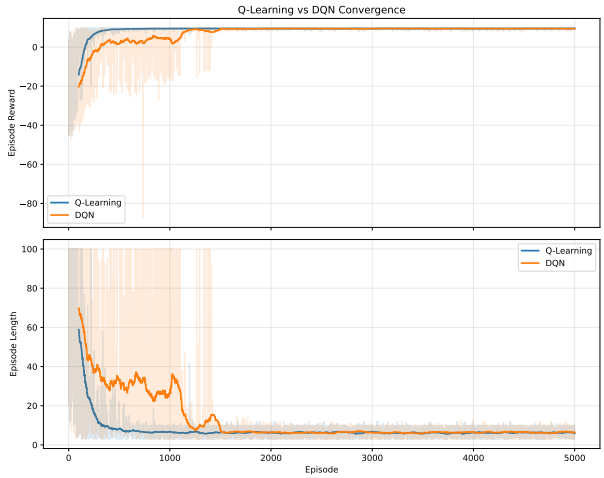

Figure 2: Convergence comparison. Q-learning (blue) converges faster than DQN (orange) on the 7×6 grid. Moving average window = 100 episodes.

Parameter sensitivity. Figure 3 shows how Q-learning convergence responds to changes in three key parameters.

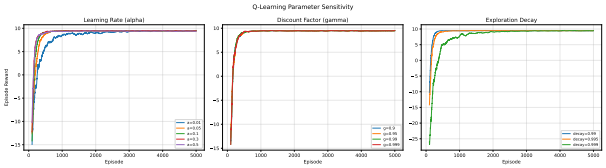

Figure 3: Q-learning parameter sensitivity. Left: learning rate α. Center: discount factor γ. Right: exploration decay rate.

Learning rate (α): Values of 0.1 and 0.3 converge fastest. $\alpha = 0.01$ learns too slowly, while $\alpha = 0.5$ converges quickly but with higher variance in episode rewards.

Discount factor (γ): All tested values (0.9, 0.95, 0.99, 0.999) produce optimal policies. However, $\gamma = 0.99$ provides the clearest value gradients toward distant goals. Lower discount factors undervalue long paths, which matters for spots like P1 that require 10 steps.

Exploration decay: A decay of 0.99 converges earliest but risks insufficient exploration of the full state space. A decay of 0.999 explores more thoroughly at the cost of slower convergence. The default 0.995 balances the two.

DQN multi-run reliability. Figure 4 shows DQN learning curves across 5 independent runs with different random seeds. All runs converge to optimal performance, with tight variance after approximately 4,000 episodes. This confirms DQN is not overly sensitive to initialization.

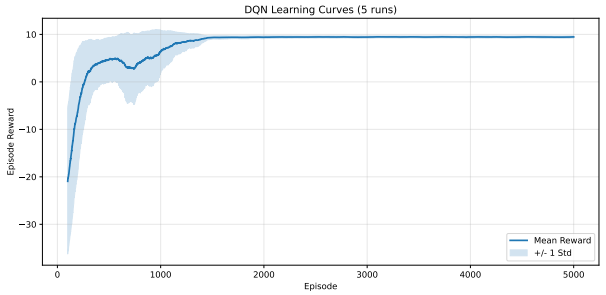

Figure 4: DQN learning curves across 5 independent runs. Shaded region shows ± 1 standard deviation.

Problem 2c: Enhancement — Larger Parking Lot

To test how the algorithms scale with complexity, we extended the environment to a 10×12 grid with 32 parking spots, up from the original 8. This increases the state space from 42 cells to 120 cells and quadruples the number of goals the agent must learn to reach.

Layout generation. The larger grid is generated procedurally using the same pattern as the base layout: two driving lanes at the top, then repeating blocks of parking row / barrier row / parking row, with a driving lane and entrance at the bottom. The barrier spans all columns except the rightmost, leaving a single passage on the right side. Parking spots are placed starting at column 1, centered within the available space.

Training adjustments. Episode counts were scaled linearly with the ratio of state-space sizes: $(10 \times 12)/(7 \times 6) \approx 2.86\times$. So tabular methods trained for approximately 14,300 episodes and DQN for approximately 8,600 episodes. No other hyperparameters were changed.

Results. Table 2 compares the three algorithms on both grid sizes.

Table 2: Scaling comparison: 7×6 (8 spots) vs. 10×12 (32 spots)

Grid	Method	Success	Avg Steps	BFS Avg
7×6	Q-Learning	100%	6.3	6.3
7×6	Double Q	100%	6.3	6.3
7×6	DQN	100%	6.3	6.3
10×12	Q-Learning	100%	11.4	11.4
10×12	Double Q	100%	11.4	11.4
10×12	DQN	100%	11.4	11.4

All three algorithms achieve 100% success with BFS-optimal paths on the larger grid. The tabular methods scale gracefully because they can represent each state independently—the Q-table simply grows from $\sim$1,300 entries to $\sim$15,400 entries. DQN's network does not grow at all; the same 17,540-parameter network handles both grids because it generalizes from the normalized position encoding. This is the practical advantage of function approximation: memory stays constant regardless of state-space size, while tabular memory grows linearly.

Adaptations. Two adaptations were made for the larger grid: (1) the maximum episode length was increased from 100 to 120 steps to accommodate longer paths, and (2) training episodes were scaled proportionally. No changes to the algorithms themselves, hyperparameters, or network architecture were required. This suggests the implementations are reasonably general.

DreamerV3 on the Parking Lot

DreamerV3 (Hafner et al. 2023) was trained on the same 7×6 parking lot to compare model-based reinforcement learning against the model-free methods described above. DreamerV3 is a model-based algorithm that learns a world model from image observations and trains its policy in "imagination" rather than through direct environment interaction.

Environment Adaptation

The parking lot was wrapped as a Gymnasium environment producing 64×64 RGB images. Six distinct colors encode the grid: dark gray for roads, medium gray for barriers, blue for non-goal parking spots, red for the goal spot, amber for the entrance, and green for the agent. The goal is communicated entirely through the image—the agent must learn that the red cell is the target.

Training Configuration

- 100,000 environment steps with 4 parallel environments

- Train ratio 512 (512 dream steps per real environment step)
- Discount factor 0.99
- 64×64 image observations
- Evaluation every 5,000 steps (10 episodes per evaluation)
- Episode time limit: 100 steps

Results

DreamerV3 reaches near-optimal performance quickly—by step 22,500, eval return peaks at 9.7 (corresponding to $\sim$4 steps, near BFS-optimal for nearby goals). However, performance oscillates throughout training rather than converging to a stable policy. The full evaluation trajectory is shown in Table 3.

Table 3: DreamerV3 evaluation returns (all 21 evaluations)

Step	Return	Step	Return
2,500	-65.3	57,500	-6.0
7,500	-10.0	62,500	$+1.5$
12,500	$+1.8$	67,500	$+5.7$
17,500	-0.2	72,500	$+1.7$
22,500	$+9.7$	77,500	$+9.4$
27,500	$+9.6$	82,500	$+1.9$
32,500	$+5.8$	87,500	$+5.5$
37,500	$+5.5$	92,500	$+1.8$
42,500	$+9.4$	97,500	$+1.5$
47,500	$+5.7$	102,500	$+8.6$
52,500	$+1.8$		

Peak return: $+9.7$ at step 22,500. **Last 5 average:** $+3.9$. Compare this to the model-free methods, which all achieve a stable $+9.1$ to $+9.8$ return after convergence.

Analysis

The returns cluster into three bands: near-optimal ($+8.6$ to $+9.7$, appearing 6 times), moderate ($+5.5$ to $+5.8$, appearing 5 times), and poor (-0.2 to $+1.9$, appearing 8 times). This suggests the agent solves nearby goals (P7, P8 with 3–4 BFS steps) reliably but struggles with distant goals (P1, P2 with 9–10 steps) that require longer navigation sequences around the barrier.

The core difficulty is representational. DreamerV3 receives the goal only as a colored pixel in the image. All 8 goals produce nearly identical images—the only difference is which of 8 small cells is red instead of blue. The tabular methods receive the goal ID as an explicit state variable, giving them unambiguous goal information. DreamerV3's world model must learn to distinguish which cell is red, associate that with different navigation strategies, and plan multi-step routes accordingly—all from a 64×64 image where the distinguishing feature occupies roughly 2% of the pixels.

Model-based RL is designed for environments where real experience is expensive and imagination can substitute. The parking lot is the opposite: episodes are 3–10 steps, the state space has $\sim$29 walkable cells, and tabular methods exhaust the space in under 2,000 episodes. There is no advantage to building a world model when direct experience is this cheap.

References

Hafner, D.; Pasukonis, J.; Ba, J.; and Lillicrap, T. 2023. Mastering Diverse Domains through World Models. *arXiv preprint arXiv:2301.04104*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.

Van Hasselt, H. 2010. Double Q-learning. In *Advances in Neural Information Processing Systems*, volume 23.